Steering long sessions

Cómo mantener a **Claude Code** bien orientado cuando la tarea dura horas: un refactor en una docena de archivos, una **feature** completa o una migración.

plan mode · /compact · rewind · /goal



PÍLDORAS DE PROGRAMACIÓN · 2026

replay

// 00 · definición

¿Qué es steering long sessions?

> Es el **conjunto de técnicas** para mantener a Claude Code bien orientado cuando la tarea **dura horas**.

No es un comando ni una feature aislada: es una forma de **dirigir** la sesión. Delimitas antes de arrancar, corriges en el momento correcto y, cuando puedes, sueltas el timón con condiciones claras.



steer · dirigir, llevar el timón

CUÁNDO APLICA · TAREAS QUE DURAN HORAS

Un refactor 01

En **una docena de archivos**. Muchas ediciones encadenadas donde cada una depende de la anterior.

`auth/` · 12 archivos · cada edición depende de la anterior

Una feature completa 02

Modelo, lógica, endpoints, tests. Construirla de punta a punta no cabe en una sola vuelta de **pides → revisas**.

`models + services + endpoints + tests`

Una migración 03

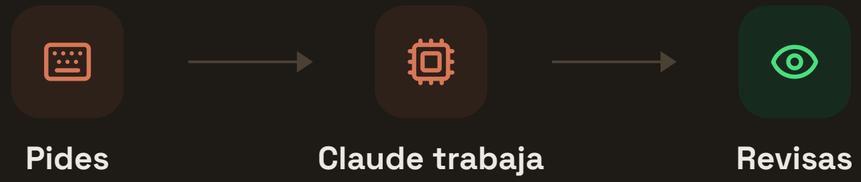
Cambiar un módulo a una **API** nueva o mover una base de datos: la sesión crece hasta que ya no cabe más contexto.

`API v1 → API v2` · hasta que compile y pasen los tests

// 01 · el problema que resuelve

Una tarea corta es simple. Una larga falla de otras formas.

tarea corta — un ciclo · un resultado



Cabe entera en tu cabeza y en el **context window**. Si algo sale mal, lo ves en la revisión y se arregla en un mensaje.

tarea larga — horas · docenas de turnos

El contexto se llena

La conversación crece hasta que **ya no cabe más**.

Claude se desvía

Toma un camino equivocado y **sigue avanzando por ahí**.

Tú corriges a mano cada paso

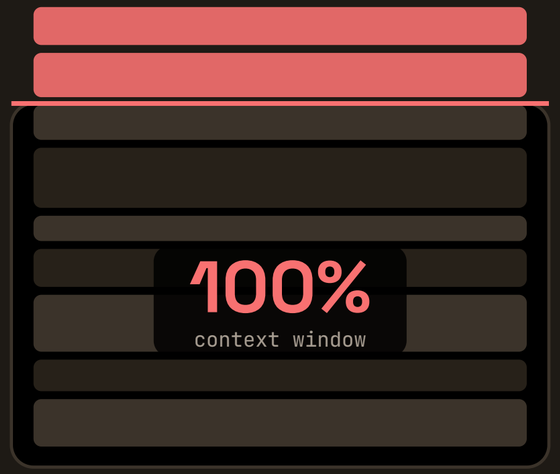
Y la sesión se alarga **todavía más**.

// 01 · el problema que resuelve

Tres formas de fallar que una tarea corta no tiene

play

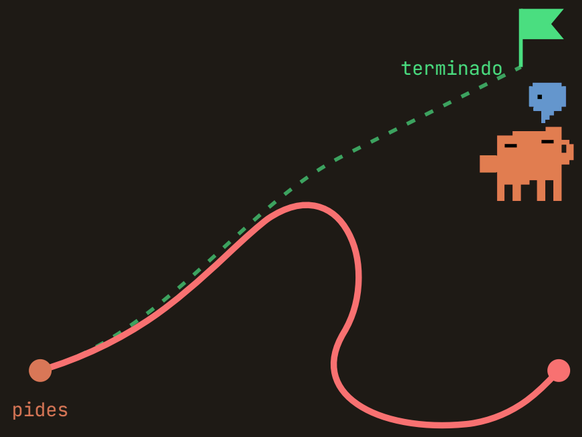
El contexto se llena



los mensajes rojos ya no caben

La conversación crece turno a turno hasta que **ya no cabe más**. Cada mensaje viejo sigue ocupando espacio.

Claude se desvía



camino equivocado · y sigue avanzando

Toma un camino equivocado y **sigue avanzando por ahí**. Cada turno lo aleja más de lo que pediste.

Corriges a mano cada paso

sesión · **3h 12m**

```
> ese no, el otro archivo
• Corregido. Ahora en services/
> faltan los tests de billing
• Agregados 4 tests…
> no rompas el type checker
• Ajustando tipos…
> vuelve a la versión anterior
• Revirtiendo… (turno 43)
```

Cada corrección es otro turno, otro mensaje en el contexto. **La sesión se alarga todavía más.**

// 01 · la conclusión del capítulo

Cuanto más corriges en el camino, más se alarga todo

LA SOLUCIÓN

No es ~~corregir más~~, sino corregir mejor y en el momento correcto.

corregir más

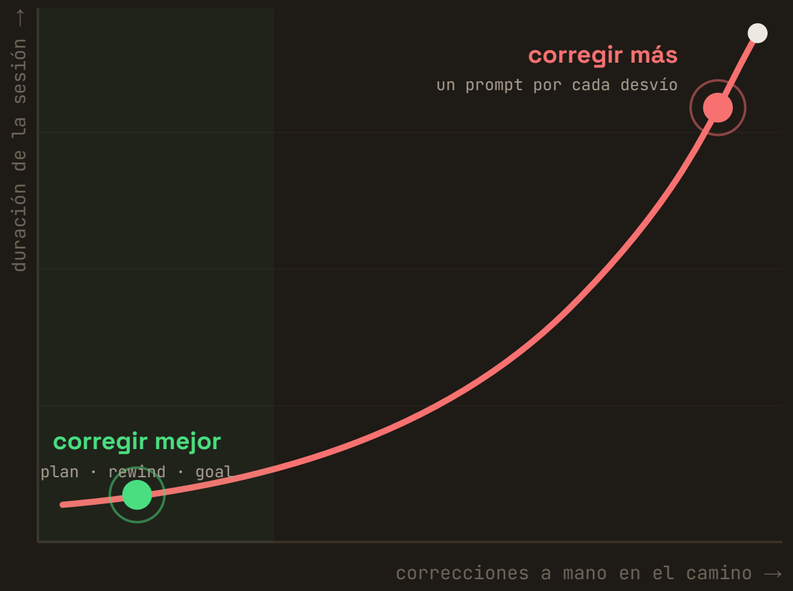
Un mensaje por cada paso torcido. Cada uno suma turnos y llena el contexto.

corregir mejor

Antes de la primera línea (plan), en el punto exacto (rewind) o con la condición clara (goal).

DURACIÓN DE LA SESIÓN VS CORRECCIONES EN EL CAMINO

play



la sesión crece con cada corrección

corregir en el momento correcto

// 02 · hábitos para sesiones largas

Cuatro hábitos, dos niveles

Dos hábitos base para no perder el rumbo, y encima de ellos dos niveles más: soltar el timón con condiciones y correr varias sesiones a la vez.

NIVEL BASE · NO PERDER EL RUMBO

01

Delimitar antes de arrancar

ANTES · PLAN MODE

Claude investiga el requerimiento y los archivos y entrega un **plan**. Lo lees a conciencia, iteras y corriges **antes de la primera línea de código**.

evita → que Claude arranque por el camino equivocado

plan mode

02

Dirigir mientras corre

DURANTE · DOS HERRAMIENTAS

/compact con instrucciones para liberar contexto sin perder lo importante, y **rewind** para volver a un checkpoint cuando Claude tomó el camino equivocado.

resuelve → el contexto lleno y el desvío a mitad de camino

/compact rewind

ENCIMA · MENOS SUPERVISIÓN

03

Soltar el timón con condiciones

CON MENOS SUPERVISIÓN

Tú defines qué es **"terminado"** o cada cuánto revisar algo, y Claude trabaja solo hasta cumplirlo.

resuelve → tener que corregir a mano cada paso

/goal /loop

04

Varios carros a la vez

EN PARALELO

Varias sesiones sobre el **mismo repo** sin que se pisen los archivos: cada una trabaja en su propio **worktree**.

resuelve → sesiones paralelas que se pisan los archivos

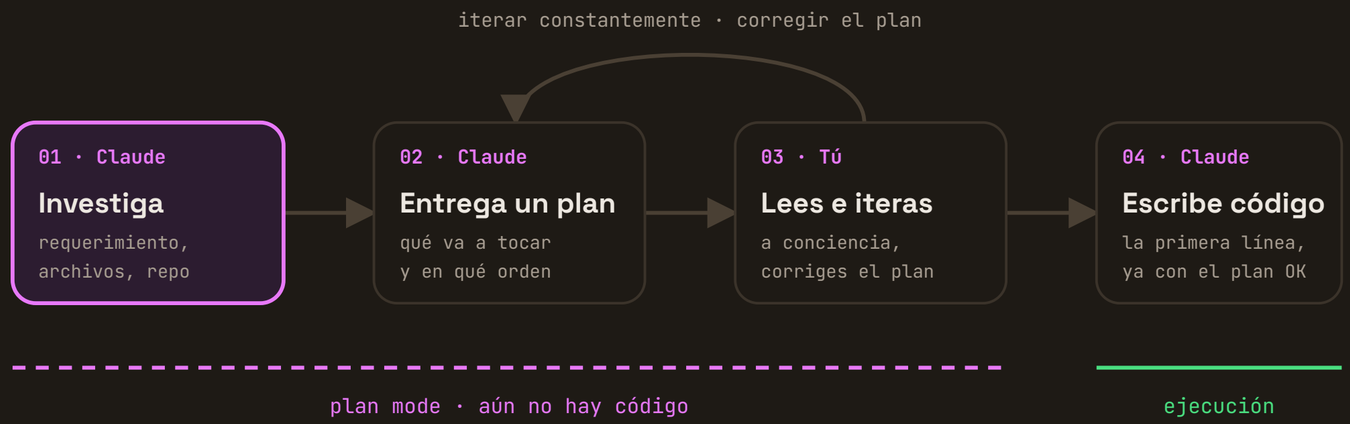
worktrees

// 02 · hábito 01 · delimitar antes de arrancar

Plan mode: corregir antes de la primera línea

1 2 3 4 ▷ ejecutar todo ↺ reset

LA PRIMERA FASE DEL DESARROLLO · QUÉ PASA ANTES DE ESCRIBIR CÓDIGO



Iterar un plan es mucho más barato que limpiar un desastre.

01

Claude investiga

En la primera fase del desarrollo, Claude investiga tu requerimiento, los archivos y el repo. Todavía no toca nada.

plan mode

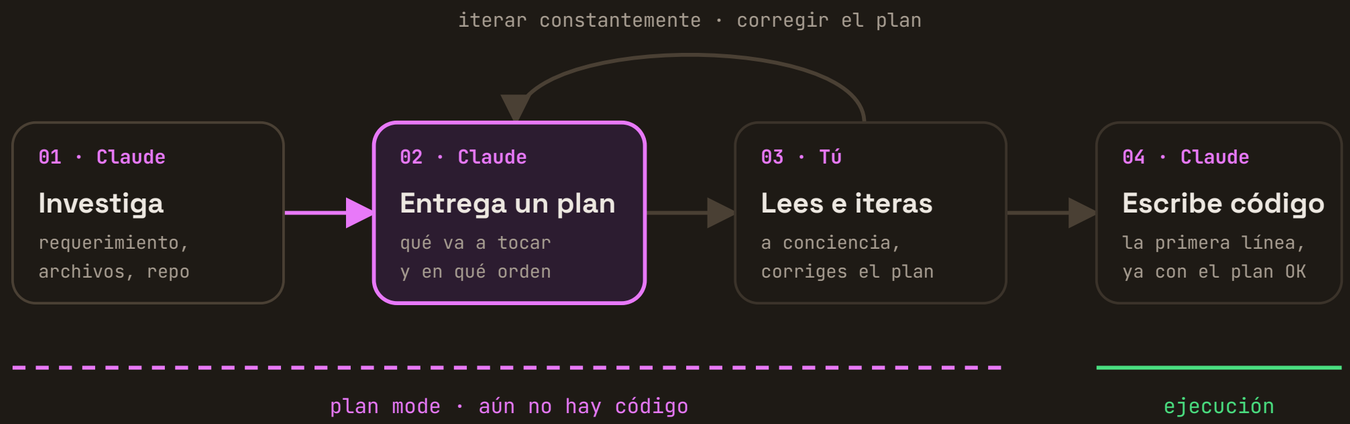
```
> migra billing/ a la API v2
• Leyendo billing/*.py …
• Revisando tests/billing …
```

// 02 · hábito 01 · delimitar antes de arrancar

Plan mode: corregir antes de la primera línea

1 2 3 4 ▷ ejecutar todo ↺ reset

LA PRIMERA FASE DEL DESARROLLO · QUÉ PASA ANTES DE ESCRIBIR CÓDIGO



Iterar un plan es mucho más barato que limpiar un desastre.

02

Claude entrega un plan

Con lo que encontró, te devuelve un plan: qué archivos va a tocar, en qué orden y qué espera que pase.

plan mode

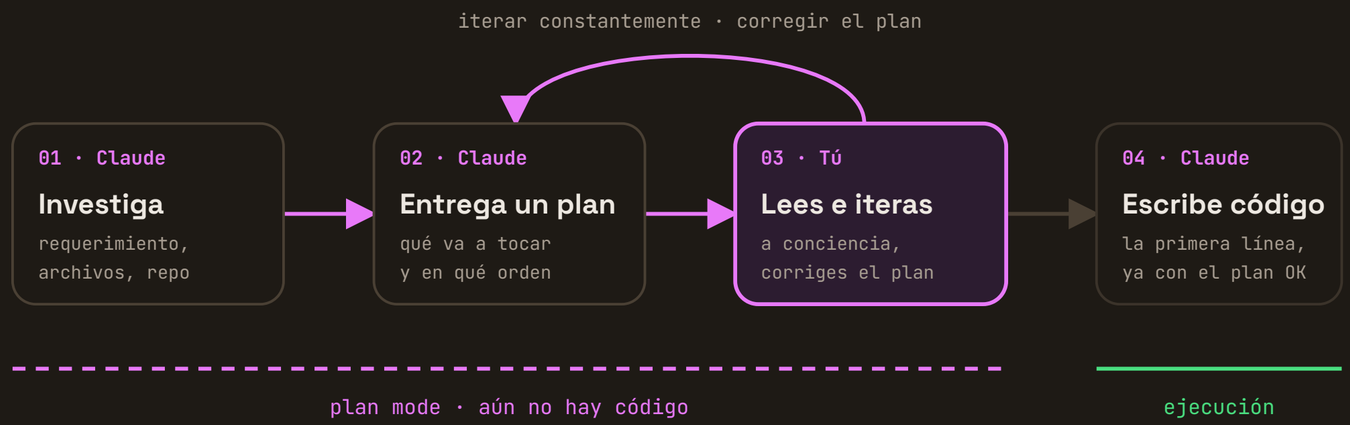
```
Plan
1. Reemplazar client v1 → v2
2. Ajustar 6 llamadas en services/
3. Actualizar tests
```

// 02 · hábito 01 · delimitar antes de arrancar

Plan mode: corregir antes de la primera línea

1 2 3 4 ▷ ejecutar todo ↺ reset

LA PRIMERA FASE DEL DESARROLLO · QUÉ PASA ANTES DE ESCRIBIR CÓDIGO



Iterar un plan es mucho más barato que limpiar un desastre.

03

Lees e iteras el plan

Es realmente importante leerlo a conciencia y entender lo que está pasando. Iteras constantemente y lo corriges hasta que refleje lo que quieres.

plan mode

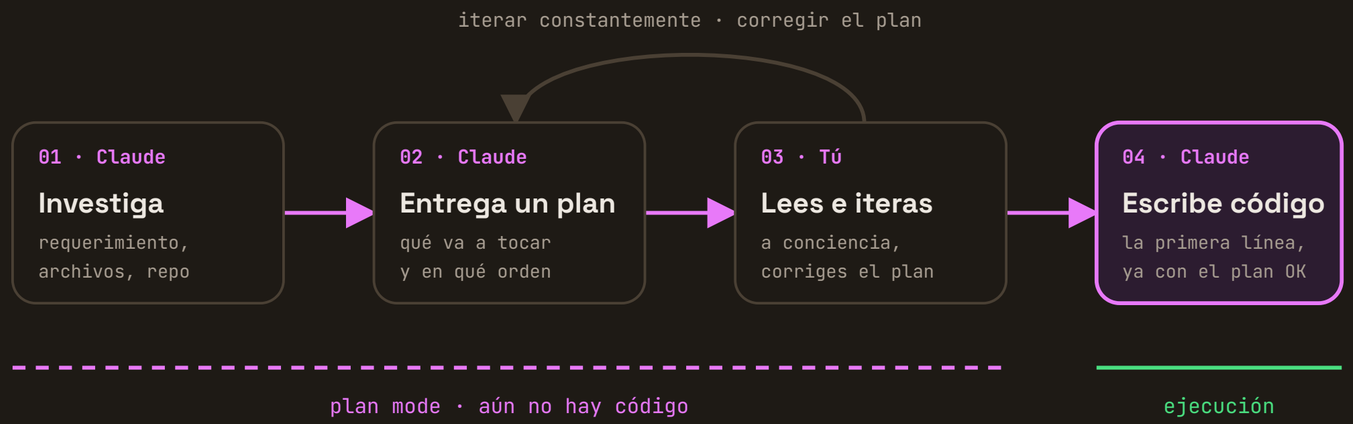
```
> no toques tests/legacy, y usa
  el wrapper de retries que ya existe
• Plan actualizado (v3)
```

// 02 · hábito 01 · delimitar antes de arrancar

Plan mode: corregir antes de la primera línea

1 2 3 4 ▷ ejecutar todo ↺ reset

LA PRIMERA FASE DEL DESARROLLO · QUÉ PASA ANTES DE ESCRIBIR CÓDIGO



Iterar un plan es mucho más barato que limpiar un desastre.

04

Recién ahí, código

La primera línea de código se escribe con el plan ya corregido. Todo lo que arreglaste aquí no lo vas a tener que arreglar después.

plan mode

```
> aprobado, ejecuta el plan
● Editando billing/client.py …
```

// 03 · hábito 02 · dirigir mientras corre

Qué es /compact

Cuando la conversación llena la **context window**, /compact resume el historial, usa el resumen como nuevo contexto y borra los mensajes viejos.

play



1 **Resume** todo el historial de la conversación.

2 Usa ese resumen como el **nuevo contexto**.

3 **Borra** los mensajes viejos. La sesión puede seguir.

// 03 · /compact · una analogía para tener en cuenta

Es como pedirle a alguien que tome notas de una reunión de tres horas

Este comando tiene un uso adicional: lo que le pides **después** de **/compact** decide qué queda en las notas.

“Toma notas.”

Notas de la reunión

- se habló del roadmap
- Juan comentó algo de la API
- hubo discusión sobre el deploy
- varios pendientes (?)
- revisar luego

~~- 40 minutos de exploración inicial~~

Anota **lo que le parezca**. Sin instrucciones, el resumen es genérico: Claude decide qué importa.

“Anota solo las decisiones sobre la API y los pendientes.”

Decisiones · API

- `/v2/payments` reemplaza a /v1
- retries con el wrapper existente

Pendientes

- migrar services/pay.py
- flag `--version` en el CLI

Te entrega **algo útil**: exactamente lo que necesitas para seguir trabajando.

Lo que escribes después de /compact le llega directo al modelo que resume

EL RECORRIDO DE TU INSTRUCCIÓN

`/compact` conserva rutas y el error literal

Tú: el comando + las instrucciones.

→

Modelo que genera el resumen

Recibe tus instrucciones como guía de **qué conservar** al resumir.

→

Resumen = nuevo contexto

Con las rutas, el comando de tests y el error **tal cual**, no parafraseados.

SIN INSTRUCCIONES

El resumen es genérico.

Claude decide qué importa y qué no. A veces acierta; a veces se lleva justo el dato que necesitabas.

SEGÚN LA DOCUMENTACIÓN

Instrucciones que le dicen a Claude qué conservar al resumir.

Esa es la definición oficial. Todo lo que va después de **/compact** es esa instrucción.

El mismo comando, tres resultados distintos

```
$/compact
```

→ RESUMEN GENÉRICO

Claude decide qué importa. Puede parafrasear o soltar justo la ruta o el error que necesitabas.

```
$/compact Conserva las rutas exactas de los archivos modificados, el comando para correr los tests y el mensaje de error actual literal. Descarta la exploración inicial del repo.
```

→ CONSERVAR + DESCARTAR + LITERAL

Dice qué guardar, qué tirar y qué debe quedar tal cual. Es la instrucción completa.

```
$/compact Enfócate en la implementación del flag --version; el debugging anterior ya está resuelto.
```

→ ENFOCAR EN LO QUE SIGUE

Cambia el centro del resumen: lo que ya se cerró sale, lo que viene queda al frente.

// 03 · /compact · anatomía de una buena instrucción

Qué conservar, qué descartar, qué debe quedar literal

Porque el resumen **tiende a parafrasear**, y una ruta parafraseada ya no es una ruta.

Qué conservar

"conserva…"

Lo que vas a necesitar en los próximos turnos: el estado actual del trabajo, no la historia de cómo llegaste.

- rutas de los archivos modificados
- el comando para correr los tests
- el error actual

Qué descartar

"descarta…"

Lo que ya cumplió su función y solo ocupa espacio. Si no lo dices, puede quedarse en el resumen.

- la exploración inicial del repo
- el debugging que ya está resuelto
- intentos que no llevaron a nada

Qué debe quedar literal

"…literal / exacto"

Datos que pierden valor si se reescriben. Pídelos tal cual, porque el resumen los va a parafrasear si no lo pides.

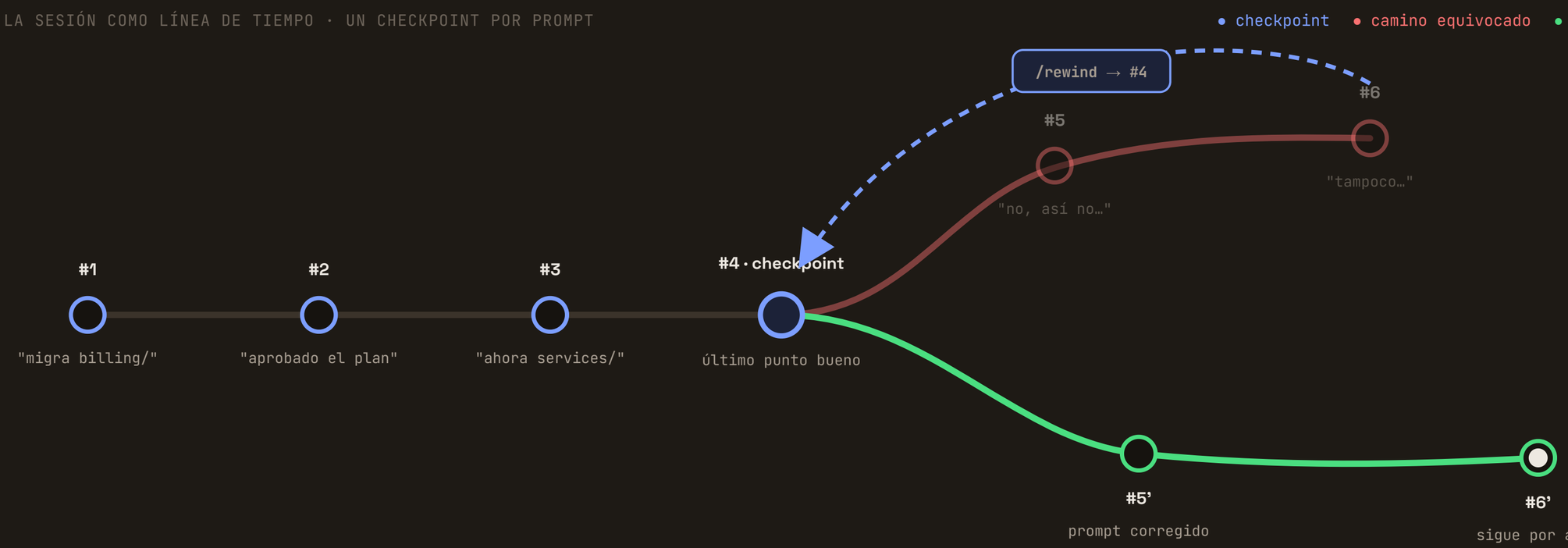
- rutas · src/billing/client.py
- errores · TypeError: retry() missing…
- números · exit code, versiones, IDs

// 04 · hábito 02 · dirigir mientras corre

Qué es rewind

Es el **"deshacer"** de Claude Code, pero a nivel de sesión. Cada prompt que envías crea un punto de control (**checkpoint**). Rewind te devuelve a cualquiera de esos puntos.

play



Cuando Claude se va por el camino equivocado, **no tienes que salir de ahí a punta de prompts**.

Vuelves al checkpoint donde todo iba bien y **arrancas de nuevo desde ahí**.

// 04 · rewind · la idea central de la lección

Corregir con más mensajes deja los intentos fallidos en el contexto

play

a punta de prompts — lo que hiciste

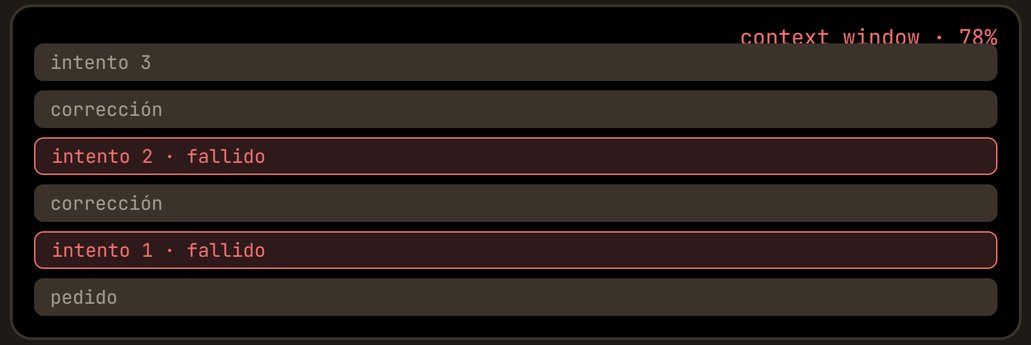
> migra el cliente de pagos a v2

• Reescribí todo billing/ con un cliente nuevo…

> no, usa el wrapper que ya existe

• Ajustado, pero cambié la firma de retry()…

> tampoco, deja la firma como estaba



Cada intento fallido **sigue ocupando contexto** y Claude lo sigue "viendo".

con rewind — lo que hiciste

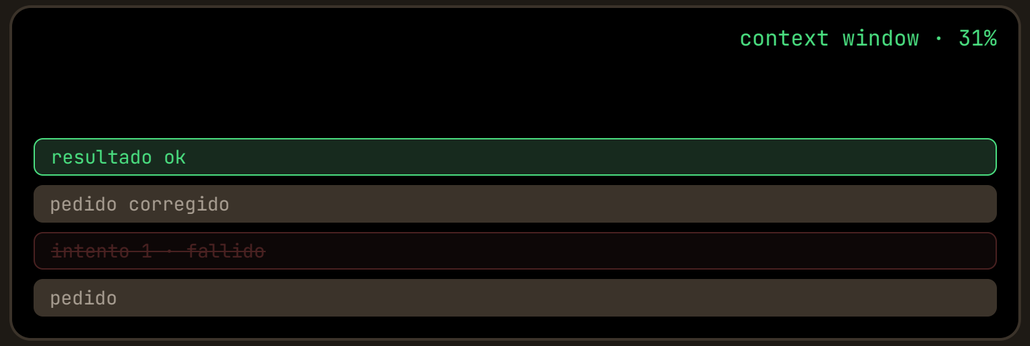
> migra el cliente de pagos a v2

~~• Reescribí todo billing/ con un cliente nuevo…~~

> /rewind → checkpoint #1 · restore code and conversation

> migra el cliente de pagos a v2 usando el wrapper de retries existente, sin cambiar firmas

• Listo: 6 llamadas migradas, tests en verde.



Con rewind esos intentos **desaparecen**: el contexto queda limpio y Claude arranca desde el punto bueno.

// 04 · rewind · cómo funciona por dentro

Claude Code guarda el estado de tu código antes de cada edición

Es automático

No tienes que activarlo ni acordarte de nada. Antes de que Claude edite un archivo, ya hay un **checkpoint** con el estado anterior.

```
> prompt #4
◆ checkpoint #4 · snapshot del código
✎ Claude edita billing/client.py
```

// un checkpoint por prompt, antes de editar

Persiste entre sesiones

Los checkpoints no mueren al cerrar la terminal: puedes usarlos en **conversaciones retomadas** días después.

```
lun sesión · checkpoints #1-#6
— cierras la terminal
jue claude --resume · #1-#6 siguen ahí
```

// los puedes usar en conversaciones retomadas

Se limpian a los 30 días

Es la limpieza por defecto, y es **configurable**. Los checkpoints son para recuperarse rápido, no para guardar historia.

```
día 0 se crea el checkpoint
día 29 todavía disponible
~~día 30 limpieza automática~~
```

// el plazo se puede configurar

// 04 · rewind · cómo se abre

Dos formas de abrirlo, y una trampa con el doble Esc

PARA ABRIR EL MENÚ

`/rewind` — El comando directo. Funciona siempre, tenga o no texto el input.

`Esc` `Esc` — Esc dos veces **con el prompt vacío**. Es el atajo rápido.

```
Rewind to a previous point · elige el checkpoint
› #4 "ahora services/" hace 12 min
#3 "aprobado el plan" hace 31 min
#2 "migra billing/" hace 48 min
#1 "migra el cliente a v2" hace 1 h
↑↓ mover · enter elegir · esc salir
```

→ primero eliges el punto, después la acción

LA TRAMPA

`Esc` `Esc` — Si el input **tiene texto**, el doble Esc **lo borra** en vez de abrir el menú.

`↑` — Nada se pierde: el texto borrado queda en tu historial y lo recuperas con la **flecha arriba**.

input	`> migra services/pay.py al wrapper`
Esc Esc	`> (borrado, no se abre el menú)`
↑	`> migra services/pay.py al wrapper`

→ el texto vuelve del historial

// 04 · rewind · las opciones del menú

Primero eliges el punto, luego la acción

· clic en cada opción

1 **Restore code and conversation**
Revierte el código y el chat

2 **Restore conversation**
Revierte solo el chat, el código se queda

3 **Restore code**
Revierte solo los archivos, el chat se queda

4 **Summarize from here**
Comprime desde ese punto hacia adelante

5 **Summarize up to here**
Comprime todo lo anterior a ese punto



CUÁNDO USARLA

Claude se desvió por completo y quieres empezar desde ese punto, como si lo de después nunca hubiera pasado.

Rewind solo revierte lo que Claude editó con sus herramientas de edición

Los cambios con bash no se rastrean

Si Claude ejecutó **rm**, **mv** o **cp**, rewind no deshace esos cambios. Solo revierte lo que editó con sus herramientas de archivos.

bash

```
$ rm -rf build/          ✗ no se rastrea
$ mv old.py new.py       ✗ no se rastrea
✎ edit client.py         ✓ checkpoint
```

Los cambios externos tampoco

Lo que tú edites a mano fuera de Claude Code, o lo que editen **otras sesiones** al mismo tiempo, normalmente no queda capturado.

fuera de Claude Code

```
✎ tú en VS Code          ✗ no se rastrea
✎ otra sesión paralela   ✗ no se rastrea
✎ esta sesión            ✓ checkpoint
```

No reemplaza a Git

Los checkpoints son para **recuperarse rápido dentro de una sesión**. El historial permanente sigue siendo Git.

dos capas

```
checkpoint   → deshacer de la sesión
git commit   → historial permanente
// uno no sustituye al otro
```

// 05 · hábito 03 · soltar el timón con condiciones

Qué es /goal

Hasta ahora todo era trabajo **hands-on**: tú mirando y corrigiendo. /goal es el primer paso hacia soltar el timón. En vez de describir los **pasos**, describes cómo se ve **"terminado"**.

hands-on · describes los pasos

1. cambia el import de client v1 a v2
2. ajusta las 6 llamadas en services/
3. corre los tests
4. si fallan, arregla y vuelve a correr
5. revisa mypy y arregla los tipos
6. vuelve a correr todo…

Tú escribes cada paso y **vuelves a escribir** cada vez que Claude termina un turno.



/goal · describes "terminado"

```
/goal pytest tests/billing termina con exit code
0 y mypy src/billing reporta 0 errores
```

- turno 1 · Claude trabaja… no cumple
- turno 2 · Claude trabaja… no cumple
- turno 3 · Claude trabaja… no cumple
- turno 4 · Claude trabaja… cumple ✓ · te devuelve el control

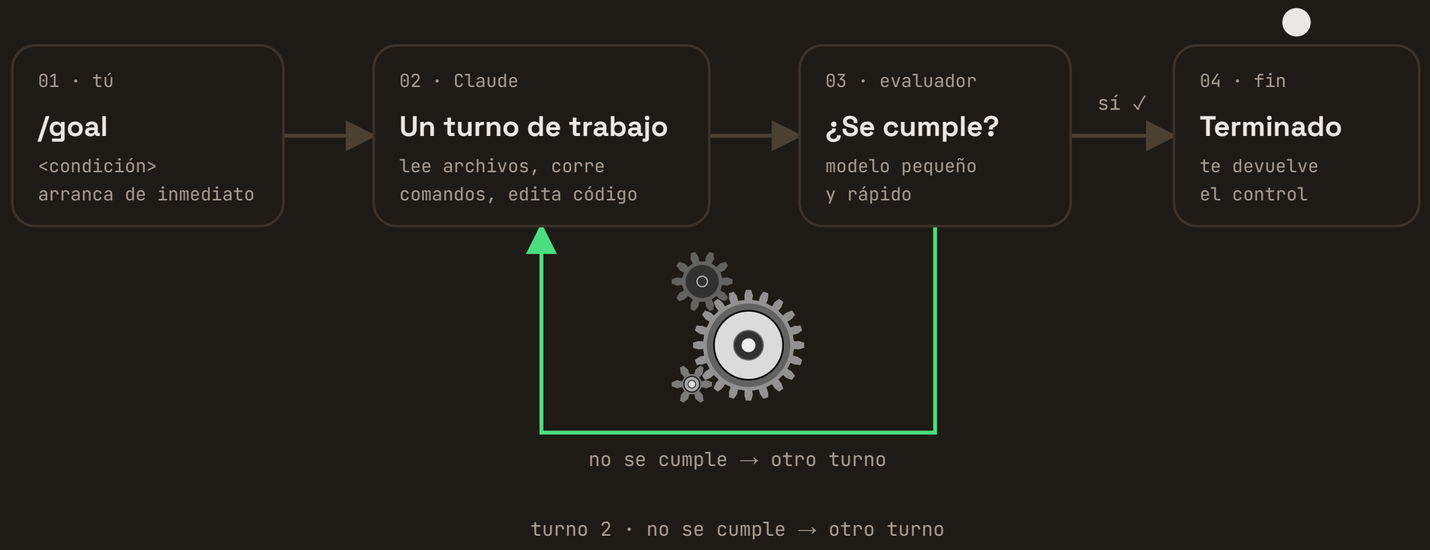
Claude trabaja **turno tras turno, sin que tú le vuelvas a escribir**, hasta que se cumple esa condición.

// 05 · /goal · cómo funciona por dentro

Un loop que solo se detiene cuando la condición se cumple

play

EL GOAL ARRANCA UN TURNO DE INMEDIATO · LA CONDICIÓN ES LA INSTRUCCIÓN



turno 2 · no se cumple → otro turno

LOS 4 PASOS

01
Escribes **/goal <condición>**. Arranca un turno de inmediato usando la condición como instrucción: **no necesitas un prompt aparte**.

02
Claude trabaja: lee archivos, corre comandos, edita código.

03
Al terminar cada turno, un **modelo pequeño y rápido** revisa si la condición se cumple. Si no, Claude arranca otro turno **en vez de devolverte el control**.

04
Se repite hasta que termina.

// 05 · /goal · un detalle que importa

El evaluador no es el mismo modelo que trabaja

Con /goal, un modelo nuevo decide si la tarea está completa, no el que la está haciendo. Así Claude **no califica su propio examen**.

EL QUE TRABAJA

Modelo principal

Lee, corre comandos, edita, decide qué hacer en cada turno. Está **dentro** de la tarea, con todo el contexto de lo que intentó.

examen · turno 3

```
[x] client v1 → v2 migrado
[x] 6 llamadas ajustadas
[x] tests corriendo…
[ ] mypy sin revisar
"creo que ya está" ← esto no vale
```

EL QUE EVALÚA

Modelo evaluador

Un modelo **nuevo, pequeño y rápido**. Solo mira la condición y la evidencia al final de cada turno: **¿se cumple o no?**

calificación · turno 3

```
pytest tests/billing → exit code 1
mypy src/billing → 0 errores
archivos fuera de src/billing → ninguno

✗ no se cumple → otro turno
```

// 05 · /goal · cómo escribir una buena condición

Las condiciones que mejor funcionan tienen tres partes

1

Un estado final medible

Algo que se pueda leer sin interpretar: un número, un código, un conteo.

- › un resultado de tests
- › un exit code de build
- › un número de archivos
- › una cola vacía

2

La prueba explícita

Cómo debe **demostrarlo** Claude. El evaluador no adivina: verifica.

- › npm test termina en 0
- › git status está limpio

3

Las restricciones

Lo que **no debe cambiar** en el camino. Sin esto, Claude puede "cumplir" rompiendo otra cosa.

- › no se modifica ningún otro archivo de tests
- › ningún archivo fuera de src/billing

```
/goal pytest tests/billing termina con exit code 0 · mypy src/billing reporta 0 errores · ningún archivo fuera de src/billing fue modificado
```

■ estado final + prueba ■ segunda prueba ■ restricción

// 05 · /goal · ejemplos para clase

De ”queda limpio” a algo que el evaluador pueda comprobar

❌ `/goal el código de billing queda limpio y bien hecho`

NO VERIFICABLE

"Limpio" no se puede medir: **el evaluador adivina**.

✅ `/goal todos los tests en src/billing pasan y el type checker reporta cero errores`

SALIDA CONCRETA

Hay un resultado que lo demuestra: tests + type checker.

✅✅ `/goal pytest tests/billing termina con exit code 0, mypy src/billing reporta 0 errores, y ningún archivo fuera de src/billing fue modificado`

ESTADO FINAL + PRUEBA + RESTRICCIÓN

Las tres partes. El evaluador solo tiene que **comprobar**, no interpretar.

// 05 · /goal · cuándo termina

El goal se limpia solo en tres casos

La condición se cumple

El evaluador comprueba la prueba que pediste y da el visto bueno. Es el final feliz.

✓ pytest → exit code 0 · goal completado

El modelo juzga que es imposible

La condición no se puede cumplir tal como está escrita. En vez de dar vueltas infinitas, el goal se detiene.

⚠ condición inalcanzable · goal detenido

Un turno falla con un error que tú tienes que arreglar

Algo que Claude no puede resolver solo. Te devuelve el control para que lo arregles.

✗ error en el turno · necesita tu intervención

cancelar

```
$ /goal clear
```

También puedes **cancelarlo tú** en cualquier momento con **/goal clear**.

goal activo ~~goal #2~~

Solo puede haber **un goal activo por sesión**. Si quieres otro, primero cierra o limpia el actual.

// 05 · /goal · cuándo usarlo

Úsalo cuando puedas describir "terminado" mejor de lo que puedes describir los pasos

sí · ejemplos de la documentación

Migrar un módulo a una API nueva

Hasta que todo compile y pasen los tests. "Terminado" es obvio; los pasos, no tanto.

`/goal el módulo compila y todos sus tests pasan`

Dividir un archivo grande

Hasta que cada módulo quede bajo un tamaño límite. Un número que el evaluador puede contar.

`/goal ningún archivo en src/ supera las 300 líneas`

no sirve para

Tareas de criterio subjetivo

"Mejora la legibilidad" no tiene una prueba que el evaluador pueda ejecutar. Eso lo juzgas tú.

✗ `/goal el código queda más legible`

Tareas cortas

Si cabe en un turno, el loop no aporta nada: pides, Claude trabaja, revisas. Como al principio.

✗ `/goal renombra la variable x a total`

// 06 · la conexión con lo anterior

Un buen goal casi siempre sale de un buen plan

Primero **plan mode** para definir qué es "terminado", después **/goal** para ejecutarlo.

ANTES · DEFINIR

plan mode

Claude investiga y entrega un plan; tú lo lees a conciencia y lo iteras. De ahí sale la definición de **"terminado"**: qué tests, qué archivos, qué no debe cambiar.

```
Plan v3
1. client v1 → v2 en billing/
2. ajustar 6 llamadas en services/
3. actualizar tests/billing
// no tocar tests/legacy · no cambiar firmas
```

el plan se vuelve condición

DESPUÉS · EJECUTAR

/goal

Con "terminado" ya definido, la condición se escribe sola: estado final, prueba y restricciones vienen directo del plan. Claude ejecuta **sin que le vuelvas a escribir**.

```
/goal pytest tests/billing termina con exit code
0, mypy src/billing reporta 0 errores, y ningún
archivo fuera de src/billing fue modificado
// tests/legacy intacto · firmas iguales
```

// 07 · recap

Corregir mejor, no corregir más

Una sesión larga no se salva a punta de prompts. Se salva delimitando antes, dirigiendo en el momento exacto y soltando el timón solo con una condición clara.

~/proyecto — claude

- plan mode — iterar antes de la primera línea
- /compact — conservar · descartar · literal
- /rewind — volver al último punto bueno
- /goal — estado · prueba · restricción



LOS PUNTOS CLAVE

Delimitar antes de arrancar

Lee el plan a conciencia e itéralo. Iterar un plan es mucho más barato que limpiar un desastre.

/compact con instrucciones

Qué conservar, qué descartar y qué debe quedar literal. Sin instrucciones, el resumen es genérico.

rewind al checkpoint

Los intentos fallidos desaparecen del contexto. Recuerda: bash y cambios externos no se rastrean, y no reemplaza a Git.

/goal para soltar el timón

Describe "terminado" con estado final, prueba y restricciones. Otro modelo evalúa: Claude no califica su propio examen.



Claude Code in Action · Píldoras de Programación pildorasdeprogramacion.com